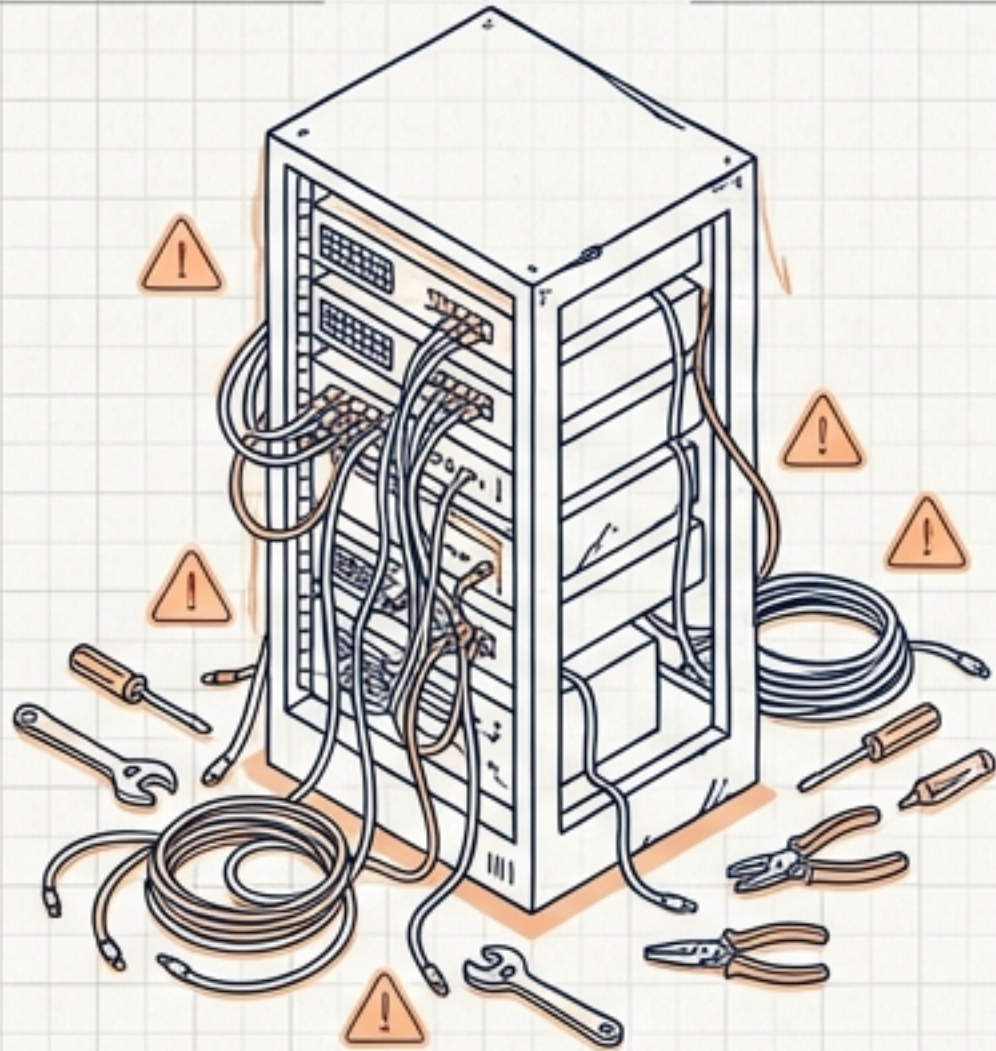


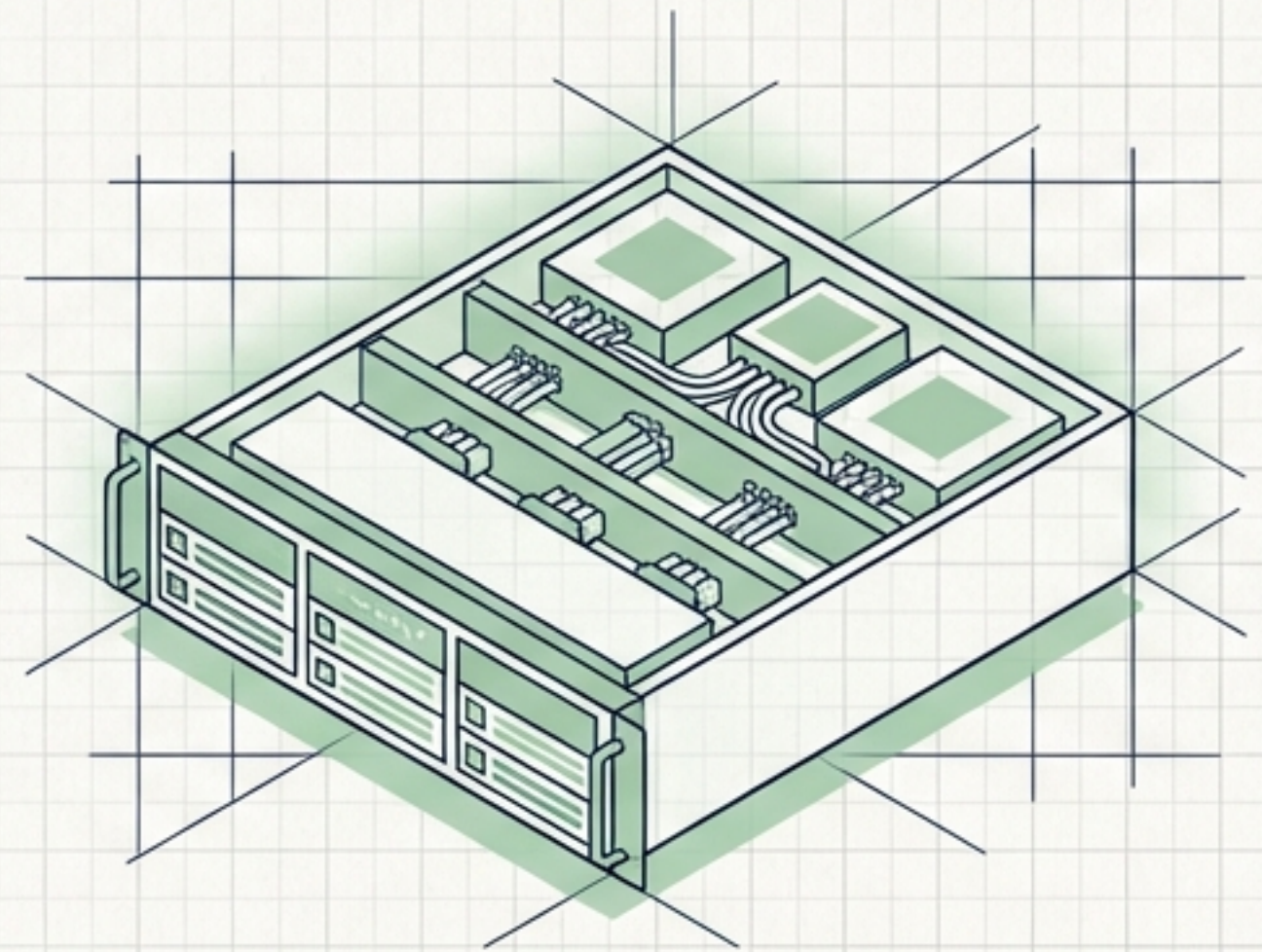
Rethinking Infrastructure: Mutable vs. Immutable

An Introduction to Automated Deployments with HashiCorp Packer

From manual configuration to baked-in perfection.



Mutable Infrastructure

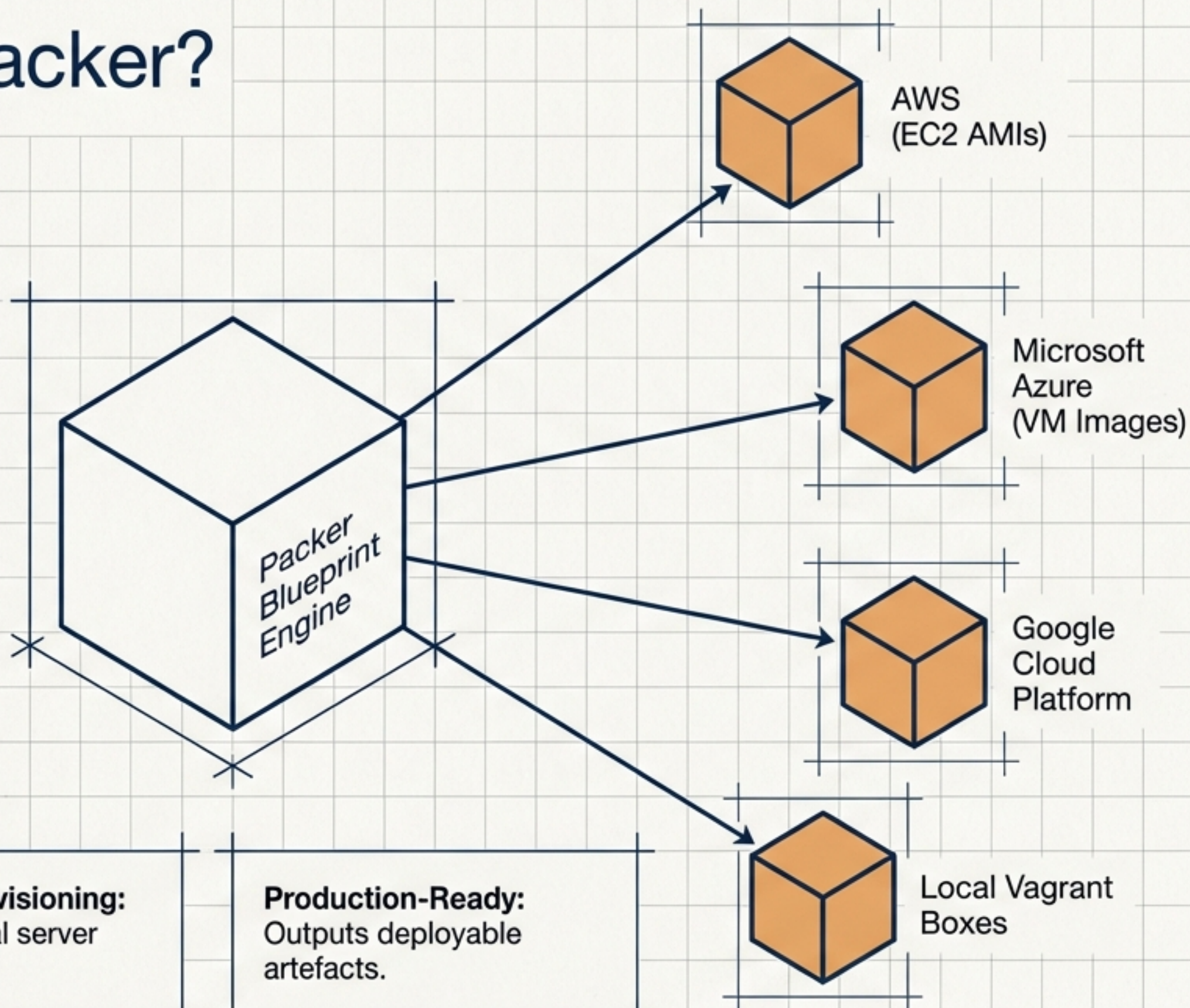


Immutable Infrastructure

What is HashiCorp Packer?

A powerful automation tool designed to create custom, identical machine images for multiple platforms from a single source configuration.

Context: The AWS Marketplace features over 1,000 pre-built AMIs. Packer enables the engineering of customised images tailored to exact application needs, replacing reliance on generic templates.



Multi-Platform Capability:
Builds for AWS, Azure, GCP, and Vagrant simultaneously.

Automated Provisioning:
Replaces manual server setups.

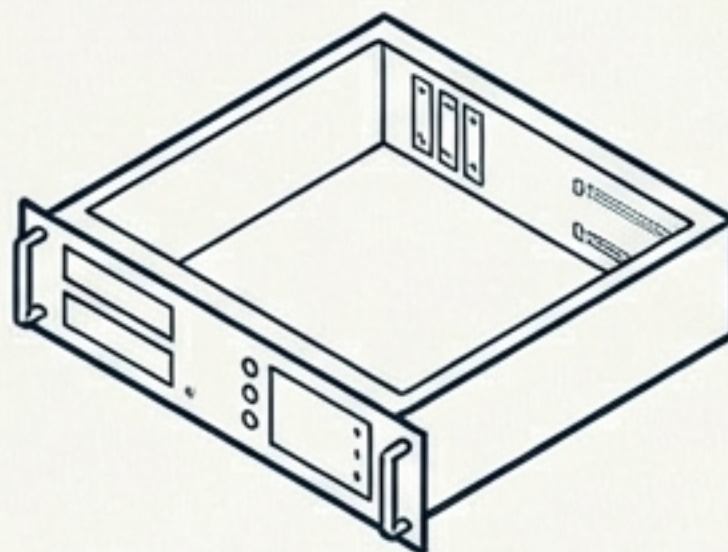
Production-Ready:
Outputs deployable artefacts.

The Mutable Infrastructure Lifecycle



Stage 1: Develop

Write application code locally.



Stage 2: Deploy

Provision a raw server (AWS EC2, Azure VM, On-Prem) with a base Linux/Windows OS.

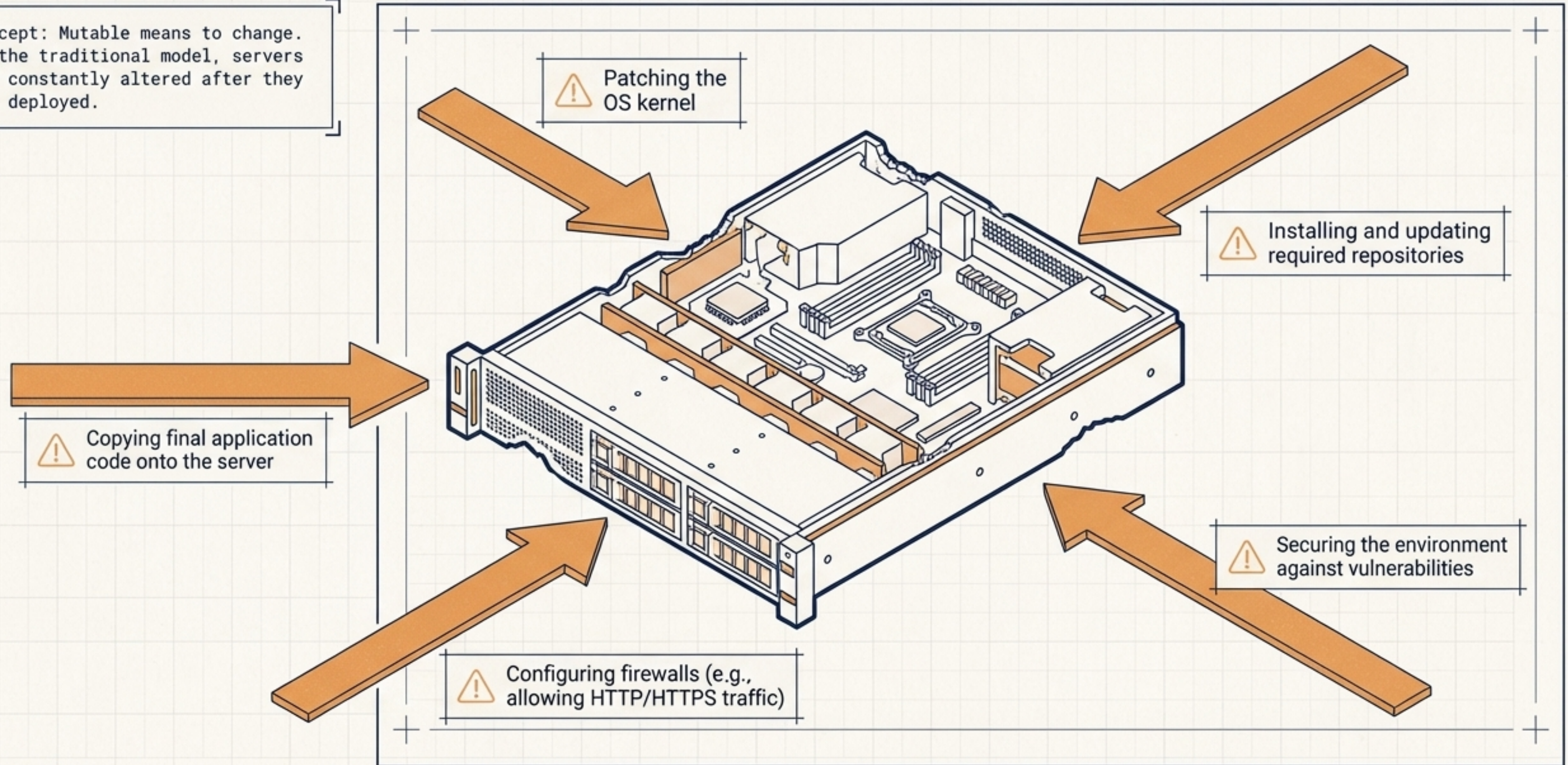


Stage 3: Configure (The Danger Zone)

Make post-deployment manual changes to the live server to make it production-ready.

"Mutating" the Live Server

Concept: Mutable means to change. In the traditional model, servers are constantly altered after they are deployed.

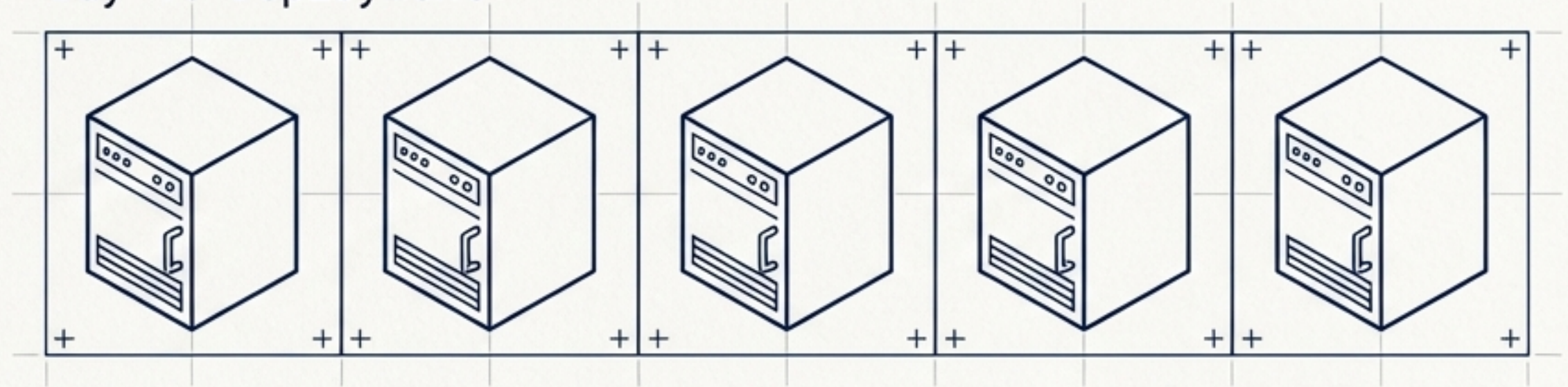


The Inevitability of Configuration Drift

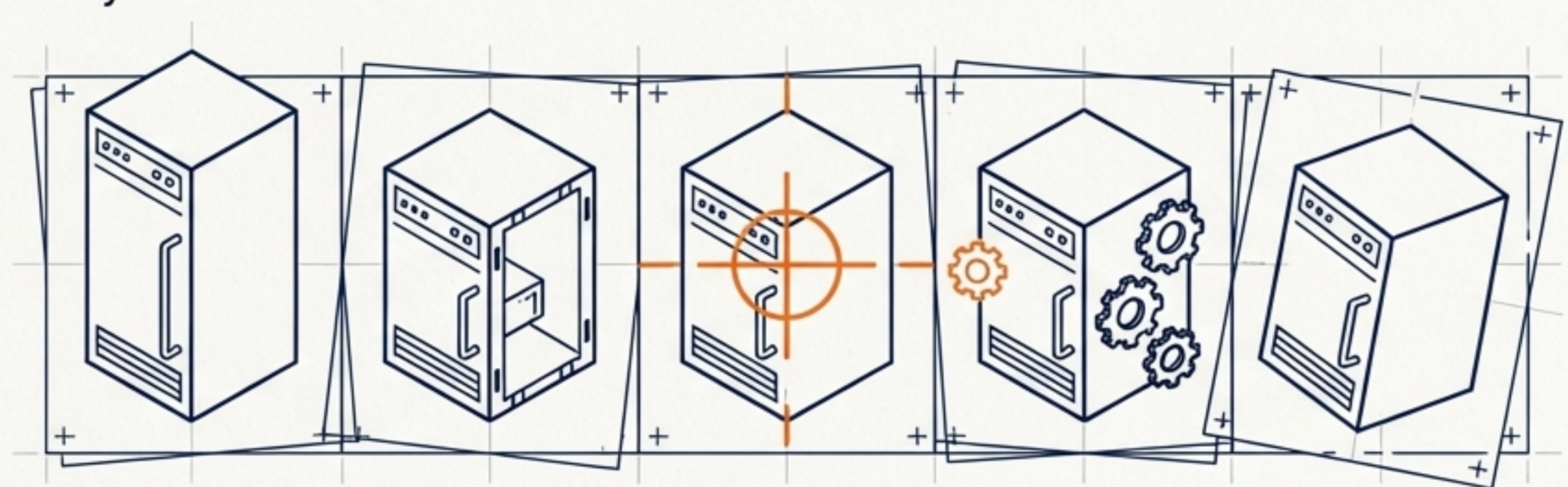
The Scale Problem:
Mutable infrastructure works
for a single server.
Managing manual configs
across hundreds of servers
inevitably leads to failure.

- The Drift Effect:
- Configurations push unevenly
across the server fleet.
 - Manual hotfixes are applied to
some servers but missed on
others.
 - Result: A fragile, mismatched
environment causing
application outages.

Day 1: Deployment



Day 100: At Scale

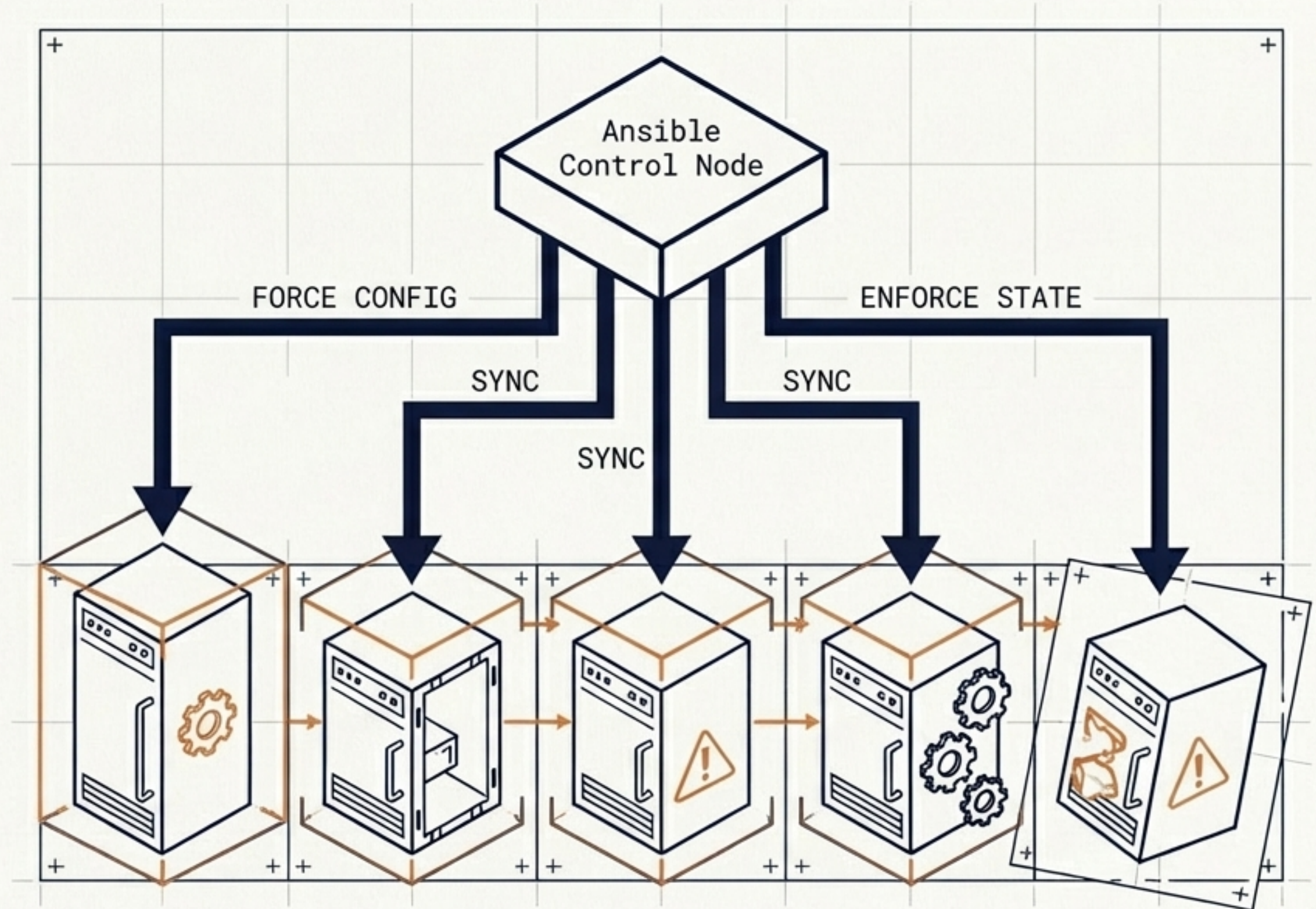


Managing the Mutations

The Interim Solution:
Configuration Management
Tools (Ansible, Chef, Puppet).

The Approach:
Use automation to
continuously force
configuration consistency
across live servers.

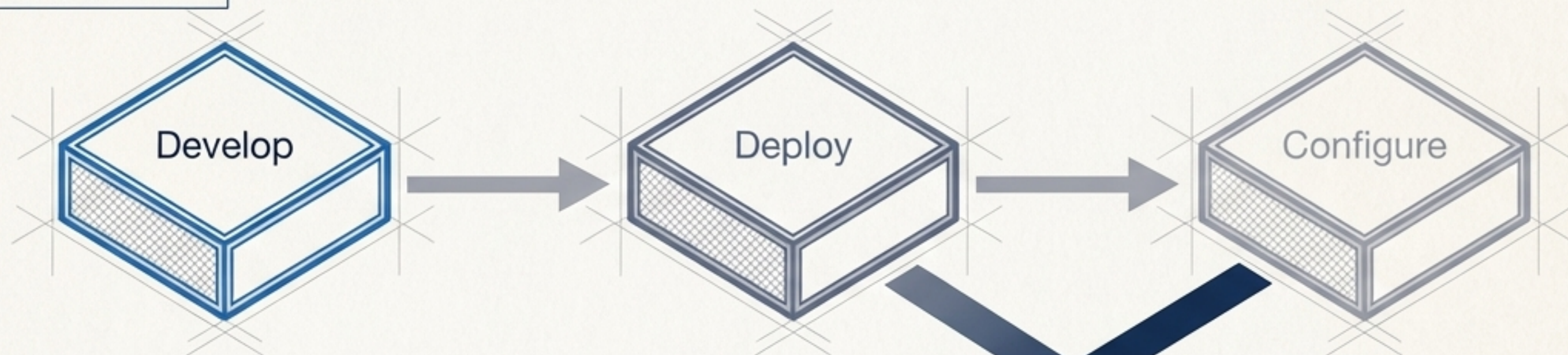
**The Verdict: A perfectly
valid, widely-used design.
However, the underlying
infrastructure remains
mutable and
inherently susceptible to
change. Is there a more
structural solution?**



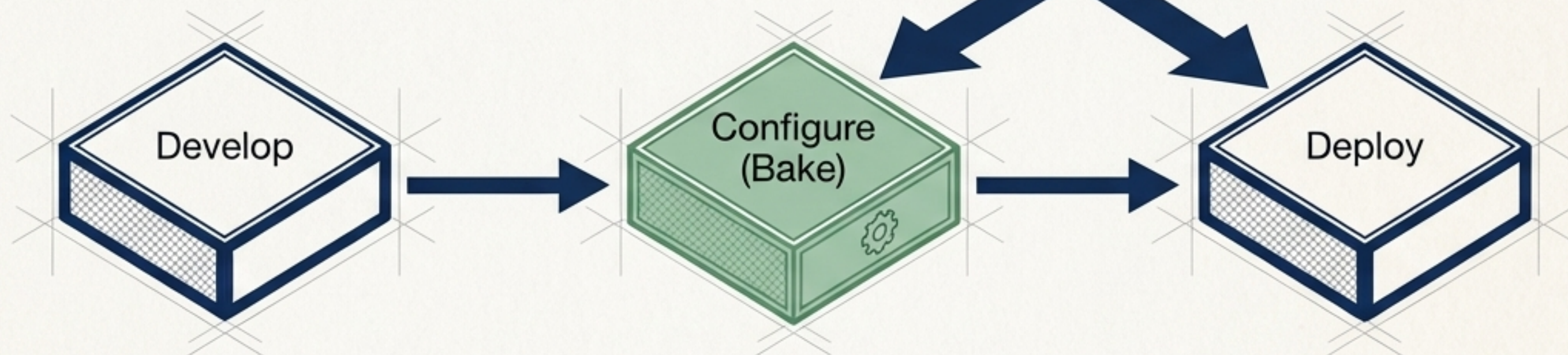
Inverting the Deployment Lifecycle

The Core Insight: What if we configure the server before we deploy it?

The Old Way
Roboto Mono



The New Way
Roboto Mono



The Rule of Immutability

Core Principle: Once a server is deployed, it is never modified.

The Unbreakable Laws

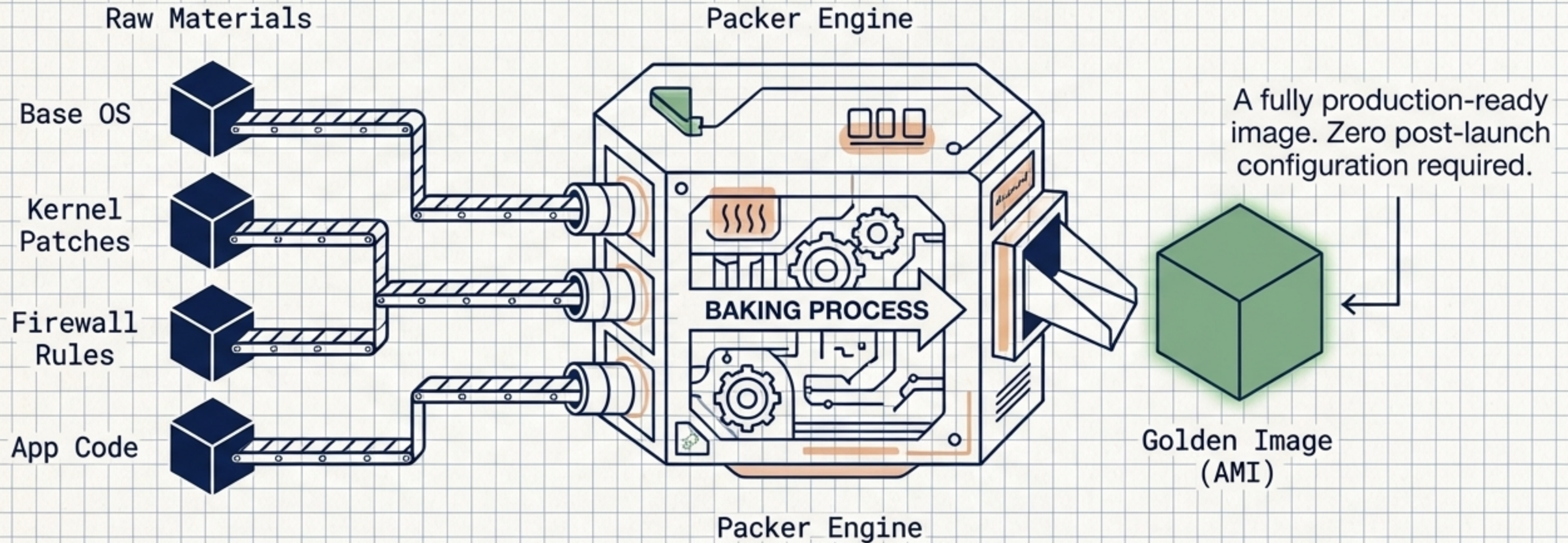
- 📁 No OS Patches.
- 📁 No Configuration Changes.
- 📁 No Application Code Updates.



The Benefit

Absolute operational consistency. The exact server state tested in staging is mathematically guaranteed to be the server running in production.

Baking the Golden Image with Packer



The Process: Packer takes all manual configuration tasks and bakes them directly into a static, deployable image prior to launch, engineering highly bespoke environments.

Paradigm Comparison Matrix

	Mutable Infrastructure	Immutable Infrastructure
Core Philosophy	Change live servers over time.	Replace live servers entirely.
Setup Sequence	Deploy, then Configure.	Configure (Bake), then Deploy.
Consistency Risk	High risk of Configuration Drift.	Zero risk (Mathematically identical images).
Primary Tooling	Ansible, Chef, Puppet.	HashiCorp Packer, Custom AMIs.

The Update Paradox

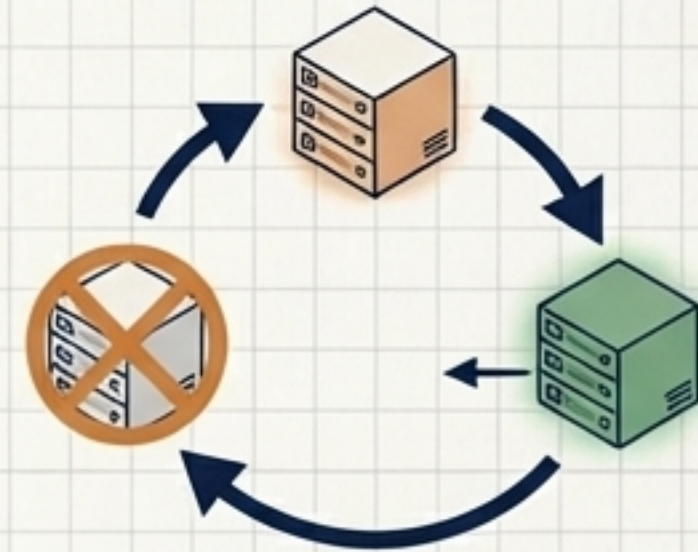
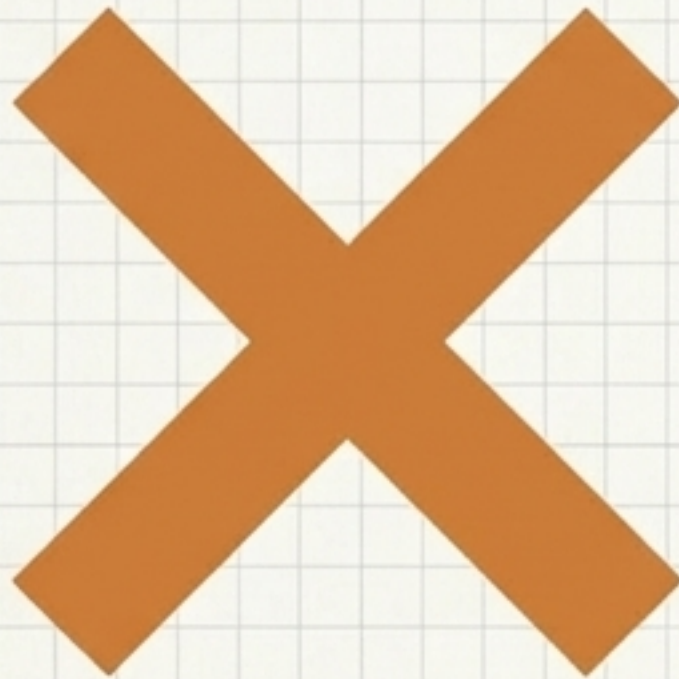
If we cannot change a deployed server, how do we push a code update or add a new software dependency?



Needs a New Dependency

Patch the live server?

Destroy and Replace

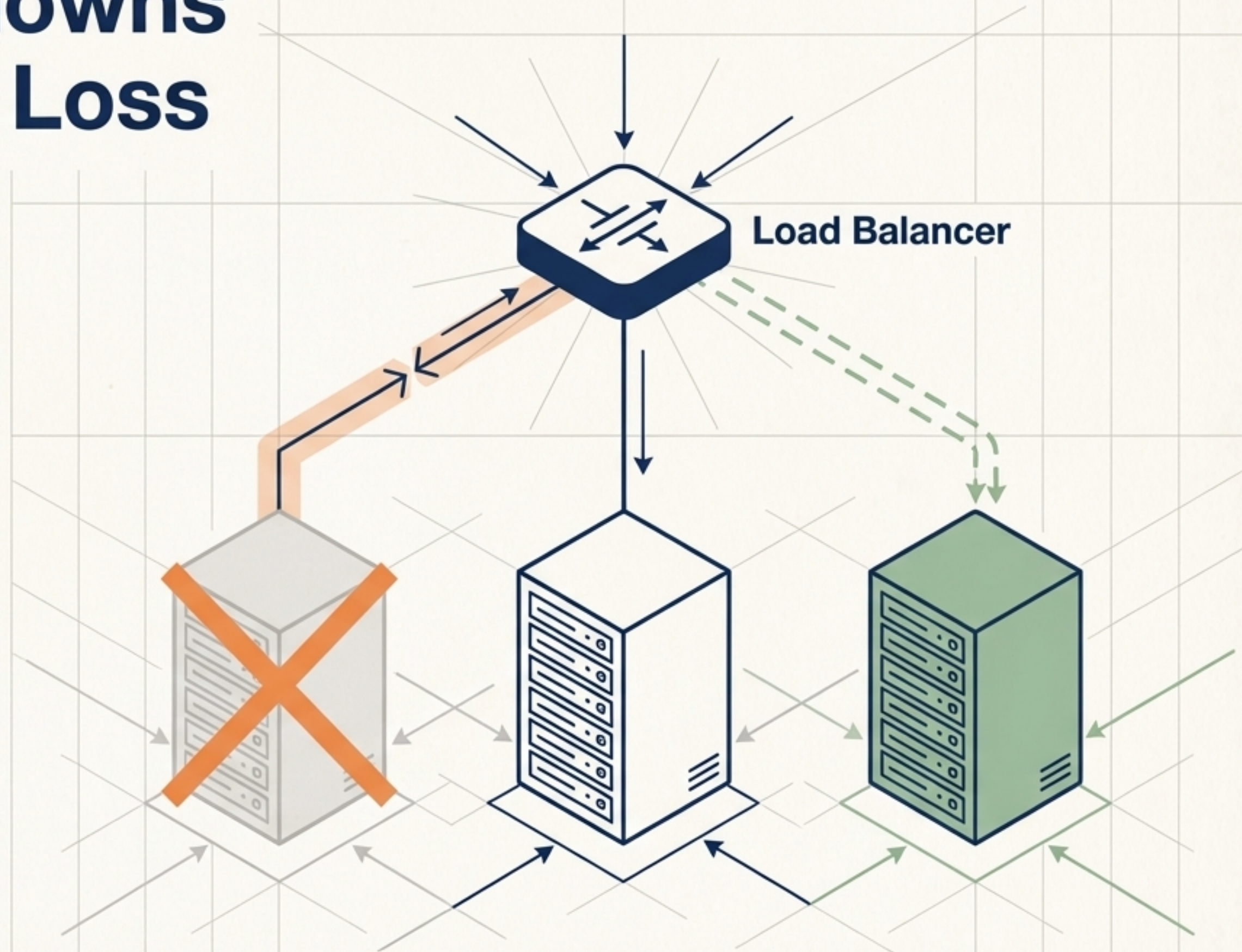


The Action: Terminate the existing live server. Immediately deploy a fresh server using a newly baked Packer image containing the updated code.

Handling Teardowns Without Traffic Loss

Zero-Downtime Architecture via Load Balancing:

1. The Load Balancer performs continuous health checks on the fleet.
2. The outdated server is marked for teardown; traffic is instantly halted to that node.
3. A fresh Packer-built server spins up from the new image.
4. The Load Balancer detects healthy status and dynamically resumes routing.



The Future is Baked, Not Sculpted

Eliminate Drift

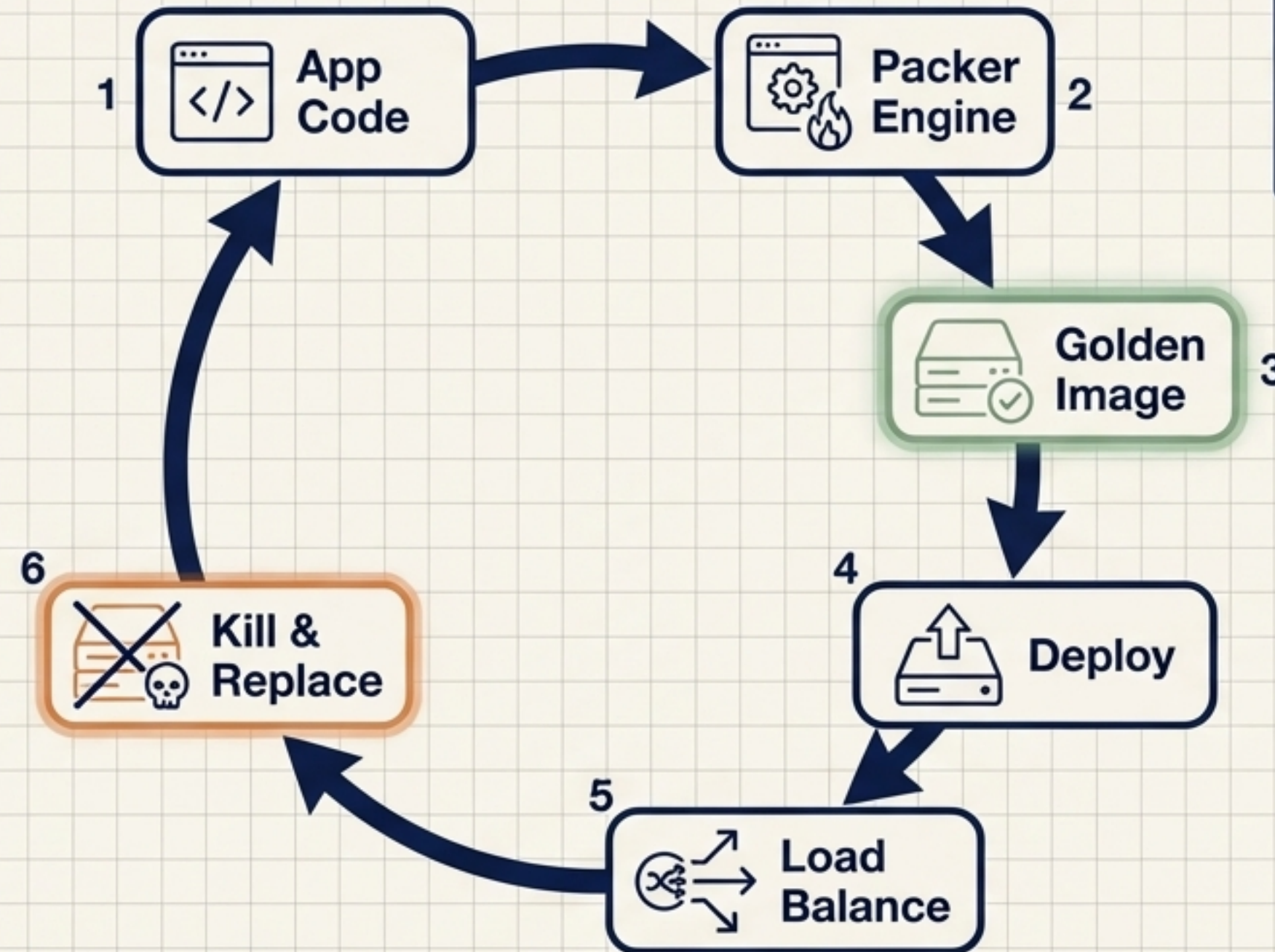
Guarantee mathematically identical environments from local staging to live production.

Accelerate Deployment

Pre-configured images boot instantly without waiting for post-launch configuration scripts.

Simplify Rollbacks

Revert to previous system states simply by deploying an older Packer image version.



The Next Step: Install HashiCorp Packer and begin engineering your first custom automated image.